Gestion automatisée et sécurisée des utilisateurs : PowerShell, Python et SQL

Projet 1SIO - Janvier 2026

Python, Powershell, SQL, Tableur, Annuaire Active Directory

Table des matières

Langages, outils et notions abordés dans le projet....3
Cahier des charges....4
Étape 00 - Constitution des équipes et méthode de travail en binôme....6
Étape 01 - Préparation des environnements de travail....7
Préparation du serveur MariaDB (Rocky 9.5)....7
Préparation de la station de travail Windows 10 (VM)....8
Utilisation de votre propre machine (PC personnel)....9
Étape 02 - Lecture et écriture dans un fichier CSV en Python....10
Étape 03 - Normalisation avancée des logins....11
Étape 04 - Création sécurisée d'une base de données et gestion des privilèges utilisateur....13
Étape 05 - Intégration d'une station Windows 10 et extraction d'informations Active Directory....14
Phase 5.1 : Intégration d'une station Windows 10 au domaine....14
Phase 5.2 : Extraction des informations de l'OU Perpignan....16
Phase 5.3 : Extraction des informations de l'OU Toulouse....18
Étape 06 - Renforcement de la politique de mots de passe....20
Phase 6.1 : Génération de mots de passe sécurisés avec un tableur....20
Phase 6.2 : Génération de mots de passe robustes avec un script Python....20
Phase 6.3 : Validation du respect des contraintes d'une liste de mots de passe....21
Étape 07 - Génération des logins utilisateurs pour MariaDB....22
Étape 08 - Création du script SQL pour utilisateurs et bases MariaDB....23
Étape 09 - Création du script SQL de suppression utilisateurs et BDD....24
Étape 10 - Rédaction d'un mode opératoire pour l'administrateur système....25
Étape 11 - Création de la base de données centralisée pour les RH....26
Étape 12 - Automatisation de l'attribution des droits aux utilisateurs Toulouse....28
Étape 13 - Renforcement de la sécurité des mots de passe dans la base de données....29
Phase 13.1 : Création d'un script Python pour hacher les mots de passe avec bcrypt....29
Phase 13.2 : Mise à jour de la base de données....30
Phase 13.3 : Vérifications....30
Étape 14 - Validation de l'authentification des utilisateurs Toulouse....31
Étape 15 - Évolution 1 – Paramètres en ligne de commande....32
Étape 16 - Évolution 2 – Mise en œuvre de "patterns" pour créer ou supprimer les utilisateurs....33

Étape 17 - Évolution 3 – Exécution automatique du script SQL.....35
Étape 18 - Évolution 4 – Exécution directe des requêtes SQL depuis Python.....36
Étape 19 - Évolution 5 – Mise en œuvre d'une interface graphique.....37
ANNEXE 1 - Notions de base sur Active Directory (AD) et les Unités d'Organisation (OU).....38
Qu'est-ce qu'un annuaire Active Directory (AD) ?.....38
À quoi sert Active Directory ?.....38
Qu'est-ce qu'une Unité d'Organisation (OU) ?.....38
Structure d'Active Directory.....38
Exemple concret.....39
Pourquoi est-ce important pour ce projet ?.....39
ANNEXE 2 - Exemple d'importation de données avec LOAD DATA INFILE.....40
ANNEXE 3 - Extraits de contenu des différents fichiers CSV.....42

Langages, outils et notions abordés dans le projet

Langages et technologies utilisés :

1. **PowerShell** :
   - Extraction des données depuis **Active Directory (AD)**.
   - Manipulation des **Unités d'Organisation (OU)**.
   - Génération de fichiers **CSV** pour les utilisateurs.
2. **Tableur** (LibreOffice Calc/Excel) :
   - Génération de mots de passe robustes selon des règles strictes.
3. **Python** :
   - Lecture et écriture dans des fichiers **CSV**.
   - Normalisation des noms et prénoms pour générer des **logins**.
   - Génération de **scripts SQL** pour la création et la suppression des utilisateurs et bases de données.
   - Génération de **mots de passe sécurisés** selon des règles strictes.
4. **SQL (MariaDB)** :
   - Création et suppression de **bases de données**.
   - Création et suppression d'**utilisateurs**.
   - Gestion des **privilèges** (accès restreints aux bases de données).
   - Exécution de scripts SQL pour automatiser les opérations.

Notions Clés Abordées :

1. **Active Directory (AD)** :
   - Structure des **domaines** et des **Unités d'Organisation (OU)**.
   - Extraction des informations utilisateurs via PowerShell.
   - Intégration d'une station Windows 10 au domaine.
2. **Sécurité et bonnes pratiques** :
   - Génération de **mots de passe robustes** (contraintes strictes).
   - Application du **principe du moindre privilège** pour les utilisateurs MariaDB.
   - Vérification des contraintes de sécurité (longueur, caractères spéciaux, etc.).
3. **Automatisation** :
   - Utilisation de **scripts PowerShell** pour extraire des données depuis l'AD.
   - Utilisation de **scripts Python** pour transformer et générer des scripts SQL.
   - Exécution de **scripts SQL** pour créer et supprimer des utilisateurs et bases de données.
4. **Gestion des données** :
   - Manipulation de fichiers **CSV** (lecture, écriture, transformation).
   - Normalisation des **logins** (gestion des caractères spéciaux, prénoms composés, etc.).
5. **Tests et validation** :
   - Vérification des **droits d'accès** des utilisateurs.
   - Validation des **opérations SQL** (création, suppression, modification).
   - Contrôle des **mots de passe** générés.

Ce projet permet d'acquérir des compétences en **automatisation**, **sécurité**, et **gestion des bases de données**, tout en utilisant des outils et langages variés pour une intégration complète entre **Active Directory** et **MariaDB**.

Cahier des charges

La société STESIO dispose d'un contrôleur de domaine CDSTESIO et d'un serveur de base de données WEB-BD-STESIO.

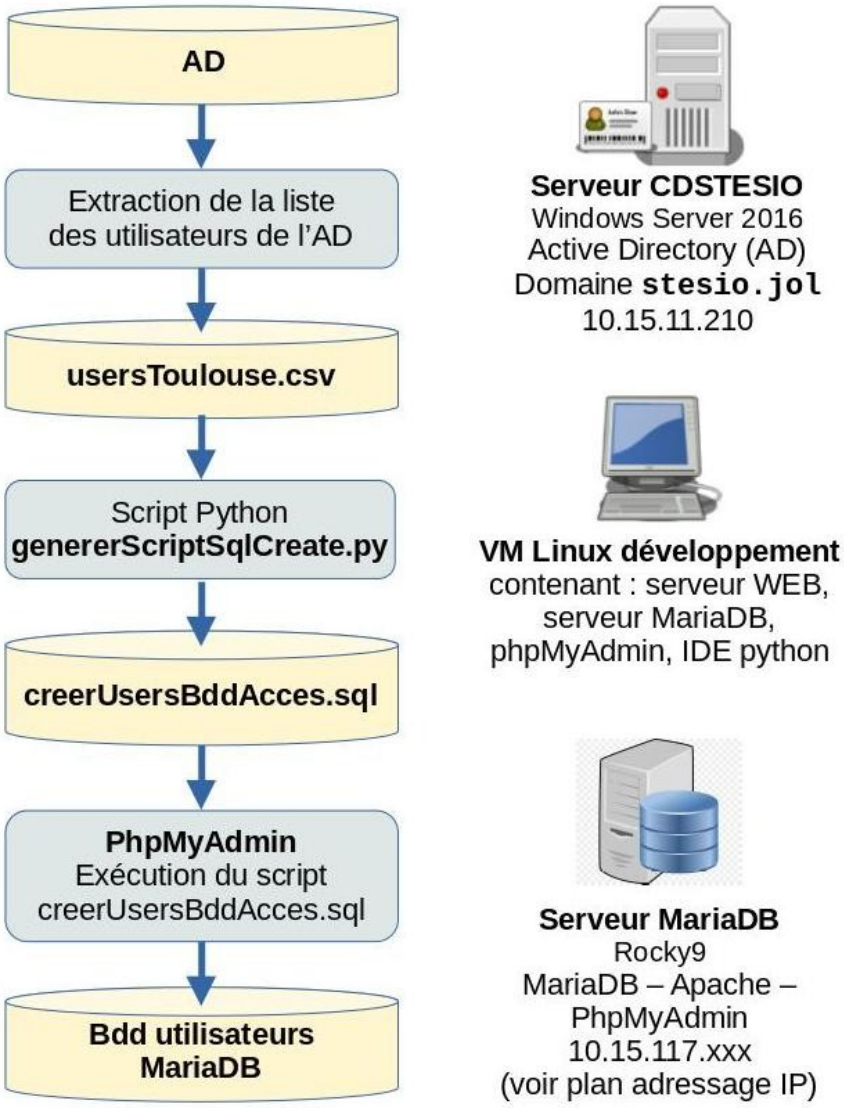
Le client souhaite que chaque utilisateur d'un de ses sites ayant un compte dans la société STESIO dispose de sa propre base de données sur le serveur de base de données avec son propre accès.

Il est possible de **récupérer la liste des utilisateurs** en interrogeant l'annuaire Active Directory (AD) du contrôleur de domaine et de la copier dans un fichier CSV (fichier texte avec séparation des informations).

Un script python générera à partir de ce fichier CSV les ordres SQL de création des comptes utilisateurs avec leur mot de passe, leur base de données et leur accès.

Le script SQL ainsi créé sera enfin exécuté dans le SGBD MariaDB pour créer les utilisateurs, les bases de données et leur accès.

Ainsi, chaque utilisateur pourra se connecter à distance à sa propre base de données (seulement la sienne) sur le serveur de base de données de la société.



Documents à produire

A chaque étape, vous devez remettre des documents dans un fichier d'archive ZIP en respectant la convention de nommage suivante : **`pyt26_bBB_eEE_nom1_nom2.zip`**

`BB` : n° binôme (01 à ...), **`EE`** : n° étape (01 à 16), **`nom1`** et **`nom2`** : noms étudiants du binôme

Exemple : `pyt26_b07_e01_morane_lefort.zip`

Un travail a été ouvert à cet effet sur la plateforme Moodle.

Matériel à disposition

Chaque étudiant dispose de plusieurs VM :

- **1 serveur MariaDB : `NOM_T_rocky9.5_PROJET_PYTHON_mariadb_251217`**
- **1 station cliente Windows 10 : NOM_T_W10_Projet_1SIO**

  Ces VM sont situées dans l'arborescence Datacenter :

  1SIO > Projet_Inter_Semestre_1SIO dans vSphere – VMWare (`https://10.15.11.4`).

  Compte utilisateur : **`btssio`** / **`btssio`** compte administrateur : **`root`** / **`joliverie`**

Évaluation

Le vendredi, chaque binôme évaluera la réalisation de deux autres binômes : la grille d'évaluation est remplie en testant le jeu d'essai et le mode opératoire fournis par le binôme évalué.

Les itérations du projet (étapes)

Étape 00 - Constitution des équipes et méthode de travail en binôme

Objectif :

Constituer un **binôme** et le déclarer aux enseignants. Le travail en binôme favorise la **répartition des tâches**, l'**entraide**, le **travail d'équipe**, la **collaboration**, et permet d'avancer plus efficacement grâce à la **complémentarité des compétences** et aux **échanges constructifs**.

Méthode de travail en binôme :

1. **Répartition des tâches** : Chaque étudiant dispose de **sa propre infrastructure** (serveur MariaDB, machine cliente Windows 10) avec ses **propres paramètres IP**. Cela signifie que chaque étudiant peut réaliser le projet de manière indépendante sur ses machines et effectuer ses propres tests.
2. **Collaboration et entraide** :
   - Un membre du binôme peut se charger d'une étape spécifique, tandis que l'autre membre travaille sur une autre étape.
   - Une fois une étape terminée, l'étudiant qui l'a réalisée **présente son travail** à son binôme.
   - Le binôme **teste la solution** sur sa propre infrastructure pour valider l'étape.
3. **Compréhension mutuelle** :
   - Chaque étudiant doit **parfaitement maîtriser chaque étape** du projet.
   - Pour cela, il peut poser des questions à son binôme ou utiliser des ressources supplémentaires (comme une IA) pour obtenir des explications complémentaires.
4. **Validation des étapes** :
   - Le binôme doit s'assurer que chaque étape est **comprise et validée** par les deux membres avant de passer à la suivante.

Répartition intelligente des tâches :

Une **mauvaise idée** serait de se répartir les tâches comme suit : un étudiant se charge de faire **toutes les étapes** et l'autre de faire **tous les comptes-rendus**. Les tâches doivent être réparties de manière **intelligente**, c'est-à-dire :

- **Alterner les étapes** : Chaque étudiant doit réaliser **au moins une étape complète** (de la réalisation au test).
- **Participer activement aux comptes-rendus** : Les deux membres du binôme doivent contribuer à la rédaction des comptes-rendus pour s'assurer que chacun comprend et maîtrise le contenu.
- **Valider ensemble** : Chaque étape doit être **testée et validée** par les deux membres du binôme sur leurs infrastructures respectives.

Étape 01 - Préparation des environnements de travail

Objectifs :

1. Mettre en service votre **serveur MariaDB** avec le bon nom de machine, les paramètres IP et le VLAN attribué.
2. Préparer votre **station de travail** pour le développement des scripts Python et la manipulation des fichiers tableurs.

Préparation du serveur MariaDB (Rocky 9.5)

Rôle du serveur MariaDB

La VM serveur MariaDB contient les services suivants :

- **Un serveur MariaDB (SGBD)** : Utile pour stocker les informations des utilisateurs et les comptes utilisateurs, et gérer les accès aux bases de données.
- **Un serveur web Apache** : Utile notamment pour utiliser l'application web **phpMyAdmin**.
- **Une application web phpMyAdmin** : Interface entre l'utilisateur et le SGBD MariaDB, permettant de gérer les bases de données de manière graphique.

Ce serveur dispose d'une **adresse IP fixe** et doit se situer dans le **VLAN** précisé dans le plan d'adressage (voir Moodle).

Travail demandé :

1. **Sélection du VLAN** :
   - Dans l'interface de gestion de votre VM (vSphere/VMWare), attribuez le **VLAN** qui vous a été assigné (ex: Plot-...).
2. **Renommage du serveur** :
   - Connectez-vous à votre serveur MariaDB (Rocky 9.5).
   - Renommez votre serveur selon le format : **`SRV-NNNNN-P`**

     (NNNNN : 5 premières lettres de votre nom, P : première lettre de votre prénom)

     Exemple : Pour **MORANE Bob** , le nom sera **`SRV-MORAN-B`**.

     Vous pouvez utiliser la commande `nmtui`

```
[btssio@SRV-DUGAS-H ~]$ hostname
SRV-DUGAS-H
```

Figure 1: Exemple de renommage du serveur mariadb

3. **Configuration des paramètres IP** :
   - Assignez l'**adresse IP** et le **masque de sous-réseau** selon le plan d'adressage.

     Adresse DNS : `10.15.11.11` pour permettre l'accès à Internet.

     Vous pouvez utiliser la commande `nmtui` pour configurer les paramètres IP et la commande `nmcli dev show` pour vérifier.

```
[btssio@SRV-DUGAS-H ~]$ nmcli dev sh
GENERAL.DEVICE:                         ens192
GENERAL.TYPE:                           ethernet
GENERAL.HWADDR:                         00:50:56:B7:E2:C7
GENERAL.MTU:                            1500
GENERAL.STATE:                          100 (connecté)
GENERAL.CONNECTION:                     ens192
GENERAL.CON-PATH:                       /org/freedesktop/NetworkManager/Active
WIRED-PROPERTIES.CARRIER:               marche
IP4.ADDRESS[1]:                         10.15.117.221/24
IP4.GATEWAY:                            10.15.117.254
IP4.ROUTE[1]:                           dst = 10.15.117.0/24, nh = 0.0.0.0, mt
IP4.ROUTE[2]:                           dst = 0.0.0.0/0, nh = 10.15.117.254, m
IP4.DNS[1]:                             10.15.11.11
```

Figure 2: Exemple de paramétrage IP. Attention, vous devez mettre l'adresse IP qui vous a été assignée dans le plan d'adressage IP (voir Moodle)

4. **Accès à Internet via Stormshield** :
   - Ouvrez un navigateur et authentifiez-vous à l'adresse : https://10.15.254.254.
   - Testez l'accès à Internet.
5. **Accès à votre espace réseau personnel** :
   - **Linux** : `smb://srvnas.jolsio.net/homes2027$/votrelogin/`
   - **Windows** : `\\srvnas.jolsio.net\homes2027$\votrelogin\`
   - Testez l'accès en transférant un fichier de votre espace réseau vers votre VM.

Préparation de la station de travail Windows 10 (VM)

Rôle de la station Windows 10 :

La station de travail Windows 10 (VM) aura pour principal rôle d'**interroger l'annuaire Active Directory du serveur STESIO situé dans le domaine stesio.jol**. Pour cela, elle devra être intégrée au domaine (voir Étape 05). Votre propre PC ne pourra pas récupérer les informations utilisateurs car il ne fait pas partie du domaine. Cette station de travail Windows 10 peut aussi servir pour les travaux demandés (scripts Python, PowerShell, tableur, etc.).

Travail demandé :

1. **Sélection du VLAN** :
   - Dans l'interface de gestion de votre VM (vSphere/VMWare), attribuez le **VLAN** qui vous a été assigné.
2. **Renommage de la station Windows 10** :
   - Ouvrez **Paramètres > Système > Informations système > Renommer cet ordinateur**.
   - Renommez votre station selon le format : **`W10-NNNNN-P`**
     (NNNNN : 5 premières lettres de votre nom, P : première lettre de votre prénom)
     Exemple : Pour **MORANE Bob** *, le nom sera* `W10-MORAN-B`.
   - Redémarrez la machine pour appliquer le changement.

3. **Configuration des paramètres IP** :
   - Assignez l'**adresse IP**, le **masque de sous-réseau**, et la **passerelle** selon le plan d'adressage.
4. **Accès à Internet via Stormshield** :
   - Ouvrez un navigateur et authentifiez-vous à l'adresse : https://10.15.254.254 **.**
   - Testez l'accès à Internet.
5. **Accès à votre espace réseau personnel** :
   - Ouvrez l'explorateur de fichiers et accédez au partage :

     `\\srvnas.jolsio.net\homes2027$\votrelogin\`
   - Testez l'accès en transférant un fichier de votre espace réseau vers votre VM.
6. **Préparation de l'IDE (Visual Studio Code)** :

   Si vous souhaitez utiliser cette station pour développer les scripts python demandés plus loin :
   - Ouvrez l'application **Visual Studio Code.**
   - Installez l'extension **Python** pour bénéficier de la complétion de code et du débogueur.
   - Testez votre environnement en exécutant un script Python simple (ex: `print("Hello, World!")`).

Utilisation de votre propre machine (PC personnel)

Si vous le souhaitez, vous pouvez utiliser **votre propre PC** pour développer les travaux et scripts demandés dans certaines étapes (scripts Python, tableur, etc.). **Votre machine doit rester dans le VLAN SIO-PEDAGO**. En aucun cas, votre propre machine ne doit être intégrée au domaine **stesio.jol** (voir Étape 05).

Livrables :

- **Un compte-rendu** incluant :
  - Un **résumé des objectifs** de l'étape.
  - Une **capture d'écran** du nom de la machine du serveur MariaDB et de ses paramètres IP.
  - Une **capture d'écran** de votre environnement Python (VSCode), avec le système d'exploitation utilisé.
  - Une **capture d'écran** confirmant l'accès à votre espace réseau personnel.
  - Une **capture d'écran** du nom de la station Windows 10 et de ses paramètres IP.

Étape 02 - Lecture et écriture dans un fichier CSV en Python

Objectif : Apprendre à lire et écrire dans un fichier CSV en utilisant Python.

Consignes :

1. Enregistrez les lignes suivantes dans un fichier nommé **testEntree.csv** :

   ```
   Prenom;NOM
   Emile;ZOLA
   Victor;HUGO
   George;SAND
   ```

2. Développez un script nommé **loginGenererBasic.py** qui :
   - Ouvre le fichier **testEntree.csv** en mode lecture.
   - Crée un fichier **testSortie.csv** en mode écriture.
   - Pour chaque ligne (en ignorant l'en-tête) :
     - Générez un login en utilisant la première lettre du prénom suivie du nom de famille, le tout en minuscules. *Exemple* : `Emile;ZOLA` → `ezola`
     - Écrivez le login obtenu dans **testSortie.csv**, avec un en-tête `login` en première ligne.

Livrables :

- Le script `loginGenererBasic.py` (commenté).
- Les fichiers `testEntree.csv` et `testSortie.csv`.
- Un compte-rendu incluant :
  - Un résumé des objectifs de l'étape.
  - Une capture d'écran des fichiers (`testEntree.csv`, `testSortie.csv`).
  - Une capture d'écran du script `loginGenererBasic.py`.

Étape 03 - Normalisation avancée des logins

Objectif : Apprendre à normaliser des noms et prénoms complexes pour générer des logins conformes aux standards d'entreprise, en gérant les caractères spéciaux, les prénoms composés et les noms à particules.

Consignes

1. Créez un fichier nommé **testEntreeSpeciaux.csv** avec le contenu suivant :

```
Prenom;NOM
Alain;Lambert
Jean-Baptiste;Le Floc'h
Sophie;O'Connor
François Xavier;DE LA VILLETANAY
Émilie;DUPONT-THIBERT
Anaïs;MARTÍNEZ-GÓMEZ
Pierre-Paul;D'ALEMBERT
Marie-Josée;ST-PIERRE
```

2. Développez un script nommé **loginNormaliser.py** qui :
   - Ouvre le fichier **testEntreeSpeciaux.csv** en mode lecture.
   - Crée un fichier **testSortieNormaliser.csv** en mode écriture.
   - Pour chaque ligne (en ignorant l'en-tête) :
     - Normalisez le prénom et le nom selon les règles suivantes :
       - Prénoms composés : Utilisez les **initiales de chaque prénom** (ex: `Jean-Baptiste` → `jb`).
       - Noms à particules/composés :
         - Supprimez les **espaces, apostrophes, tirets et caractères spéciaux** (ex: `Le Floc'h` → `lefloch`, `DE LA VILLETANAY` → `delavilletanay`).
         - Remplacez les **accents** par leur équivalent non accentué (ex: `Émilie` → `emilie`, `MARTÍNEZ-GÓMEZ` → `martinezgomez`).
       - Générez le login en concaténant les initiales du prénom et le nom normalisé, le tout en **minuscules**. *Exemples* :

Prénom;NOM	Login normalisé
Alain;Lambert	`alambert`
Jean-Baptiste;Le Floc'h	`jblefloch`
Sophie;O'Connor	`soconnor`
François Xavier;DE LA VILLETANAY	`fxdelavilletanay`
Émilie;DUPONT-THIBERT	`edupontthiber`
Anaïs;MARTÍNEZ-GÓMEZ	`amartinezgomez`
Pierre-Paul;D'ALEMBERT	`ppdalembert`
Marie-Josée;ST-PIERRE	`mjstpierre`

- **Écrivez** chaque login dans **testSortieNormaliser.csv**, avec un en-tête `login` en première ligne.

Livrables :

- Le script **loginNormaliser.py** (commenté et documenté).
- Les fichiers **testEntreeSpeciaux.csv** et **testSortieNormaliser.csv**.
- Un compte-rendu avec :
  - Un résumé des objectifs de l'étape.
  - Une capture d'écran des fichiers d'entrée/sortie.
  - Une explication des choix de normalisation (pourquoi supprimer les apostrophes, comment gérer les accents, etc.).
  - Un exemple d'exécution du script.

Étape 04 - Création sécurisée d'une base de données et gestion des privilèges utilisateur

Objectif : Créer une base de données MariaDB et un utilisateur associé en appliquant le principe du **moindre privilège**. Chaque étudiant utilisera son propre nom pour générer les identifiants.

Consignes

1. Définissez les noms suivants en utilisant **votre propre nom et prénom** :
   - **Base de données** : `db_NNNNN_P`

     Exemple : Pour **Morane Bob**, la base s'appellera `db_moran_b` (5 premières lettres du **nom** + première lettre du **prénom**, en minuscules).
   - **Utilisateur** : `pnom`

     Exemple : Pour **Morane Bob**, l'utilisateur sera `bmorane` (première lettre du prénom + nom en minuscules, sans caractères spéciaux).
2. Recherchez et exécutez les commandes SQL pour :
   - Créer votre base de données (`db_NNNNN_P`).
   - Créer votre utilisateur avec le mot de passe `P@ssw0rd`.
   - Lui attribuer **uniquement les privilèges** nécessaires sur sa propre base pour :
     - Créer, modifier et supprimer des tables et des colonnes.
     - Insérer, modifier et supprimer des données.
     - Lire les données.
3. **Testez** les droits de votre utilisateur en vérifiant qu'il peut :
   - Se connecter **uniquement** à sa base de données.
   - **Créer**, **modifier** et **supprimer** des tables et des colonnes.
   - **Insérer**, **modifier** et **supprimer** des enregistrements.
   - **Lire** les données de ses tables.
   - **Ne pas** accéder à d'autres bases de données.
   - **Ne pas** effectuer d'opérations administratives (ex: créer un autre utilisateur).
4. **Documentez** votre travail avec :
   - Les commandes SQL utilisées.
   - Les résultats des tests (captures d'écran des **succès** et des **échecs**).
   - Une justification des privilèges attribués.

Livrables :

- Un compte-rendu incluant :
  - Un résumé des objectifs de l'étape.
  - Les commandes SQL exécutées.
  - Les preuves des tests (captures d'écran commentées).
  - Une analyse des restrictions appliquées.

Étape 05 - Intégration d'une station Windows 10 et extraction d'informations Active Directory

Cette étape vise à :

1. **Intégrer une station Windows 10 au domaine** `stesio.jol` en utilisant un compte dédié.
2. **Extraire des informations utilisateurs** depuis différentes Unités d'Organisation (OU) à l'aide de scripts PowerShell.
3. **Adapter un script existant** pour répondre à des besoins spécifiques.

Phase 5.1 : Intégration d'une station Windows 10 au domaine

On souhaite maintenant **intégrer une machine Windows 10 au domaine `stesio.jol`** en utilisant un compte dédié (`binomeXX`).

Cette opération est essentielle, car une fois intégrée au domaine, cette station de travail permettra d'exécuter des scripts PowerShell à distance. En effet, une machine en dehors du domaine ne pourrait pas accéder aux ressources nécessaires, et le contrôleur de domaine refuserait toute tentative de connexion ou d'exécution de scripts.

1. **Renommer la station Windows 10**
   - **Ouvrir les paramètres système** :
     - Faire un clic droit sur **Ce PC** > **Propriétés** > **Paramètres système avancés** > **Nom de l'ordinateur** > **Modifier**.
     - Renommer la machine selon le format : **`W10-NNNNN-P`** où :
       - **`NNNNN`** : 5 premières lettres de votre **nom de famille**.
       - **`P`** : 1ère lettre de votre **prénom**.
   - **Redémarrer la machine** pour appliquer le changement.
2. **Intégrer la station au domaine**
   - **Ouvrir les paramètres système** :
     - Retourner dans **Propriétés système** > **Nom de l'ordinateur** > **Modifier**.
     - Sélectionner **Domaine** et entrer **`stesio.jol`**.
   - **S'authentifier avec le compte binôme** :
     - Utiliser le compte **`binomeXX`** (où **`XX`** est le numéro de votre binôme, ex: **`binome01`**).
     - Saisir le mot de passe **`P@ssw0rd`**
   - **Redémarrer la machine** pour finaliser l'intégration.

3. Vérifier l'intégration au domaine

- **Se connecter avec le compte binôme** :
  - Utiliser le compte **`binomeXX`** pour vous connecter à la machine.
  - Vérifier que vous pouvez accéder aux ressources du domaine.

    Pour confirmer que votre machine est correctement intégrée au domaine **`stesio.jol`**, ouvrez une **console PowerShell** et exécutez les commandes suivantes :
    - **Tester la connectivité au contrôleur de domaine** :

      `Test-NetConnection -ComputerName stesio.jol -Port 389`
      - **Résultat attendu** : `TcpTestSucceeded : True`
      - Cela confirme que votre machine peut communiquer avec le contrôleur de domaine via le port LDAP (389).
    - **Afficher les informations de l'utilisateur connecté** :

      `whoami /all`
    - **Résultat attendu** : Les informations de votre compte (`binomeXX`) et les groupes de sécurité associés doivent apparaître.

Livrables

voir à la fin de cette étape

Phase 5.2 : Extraction des informations de l'OU Perpignan

Pour mieux comprendre les notions de base sur Active Directory et les Unités d'Organisation (OU), reportez-vous à l'Annexe 1 avant de commencer les manipulations pratiques. Cela vous aidera à suivre les étapes du projet avec plus de clarté et d'efficacité.

On souhaite récupérer les informations des utilisateurs présents dans l'OU Perpignan. Pour cela, vous allez exécuter un script PowerShell fourni, conçu pour extraire et afficher les données des comptes utilisateurs de cette OU.

1. **Récupérer le script fourni**
   - Copier le script ci-dessous dans un fichier nommé **`extractUsersPerpignan.ps1`**, à placer dans **`D:\powershell`** (créer le dossier s'il n'existe pas).

```
#
# SCRIPT_NAME : extractUsersPerpignan.ps1
# AUTHOR : Joliverie
# DATE : 16/12/2025
# VERSION : 1.0
# DESCRIPTION :
#   Ce script extrait les informations des utilisateurs de l'OU Perpignan
#   et les exporte dans un fichier CSV avec les attributs suivants :
#   Prenom, Nom, Nom complet, Email, Telephone.
#   Fichier encode en UTF-8, separateur virgule, fins de ligne au format Unix (LF)
# NOTES :
#   Necessite le module ActiveDirectory.
#   Le fichier CSV sera genere dans D:\powershell\usersPerpignan.csv

# Parametres
$Server = "10.15.11.210"
$OUPath = "OU=Perpignan,OU=Finances,DC=stesio,DC=jol"
$OutputFile = "D:\powershell\usersPerpignan.csv"

# Recupere les utilisateurs et exporte en CSV
try {
    Get-ADUser -Server $Server -SearchBase $OUPath -Filter * -Properties givenName,
sn, displayName, mail, telephoneNumber |
        Select-Object givenName, sn, displayName, mail, telephoneNumber |
        Export-Csv -Path $OutputFile -Delimiter ";" -Encoding UTF8
-NoTypeInformation -Force

    # Convertit les fins de ligne en format Unix (LF)
    (Get-Content $OutputFile -Raw).Replace("`r`n", "`n") | Set-Content $OutputFile
-NoNewline
    Write-Host "Export reussi : $OutputFile" -ForegroundColor Green
} catch {
    Write-Host "Erreur : $_" -ForegroundColor Red
    Write-Host "Verifie :"
    Write-Host "1. Le chemin de l'OU : $OUPath"
    Write-Host "2. Tes droits d'acces sur ce serveur et cette OU."
    Write-Host "3. Que le module ActiveDirectory est installe."
}
```

2. **Exécuter le script**
   - **Ouvrer PowerShell en tant qu'utilisateur standard** (avec le compte `binomeXX`).
   - **Autoriser l'exécution des scripts** en tapant :

     ```
     Set-ExecutionPolicy Unrestricted -Scope CurrentUser -Force
     ```

     (Cette commande permet d'exécuter des scripts PowerShell, y compris ceux situés sur des partages réseau ou locaux.)
   - **Exécuter le script** en tapant :

     ```
     & "D:\powershell\extractUsersPerpignan.ps1"
     ```

     (Le symbole **&** en PowerShell permet **d'exécuter un script ou une commande** à partir de son chemin, plutôt que de simplement afficher le chemin comme du texte.)
3. **Vérifier le résultat**
   - Ouvrir `D:\powershell\usersPerpignan.csv` et vérifier que les données sont correctement extraites.
   - Vérifier que le fichier contient bien les colonnes : **`givenName`**, **`sn`**, **`displayName`**, **`mail`**, **`telephoneNumber`**.

usersPerpignan.csv

```
1 "givenName";"sn";"displayName";"mail";"telephoneNumber"
2 "Julie";"PRIGENT";"PRIGENT Julie";"julie.prigent@stesio.jol";"0159000373"
3 "Caroline";"PARMENTIER";"PARMENTIER Caroline";"caroline.parmentier@stesio.jol";"0159000383"
4 "Eva";"DUCLOS";"DUCLOS Eva";"eva.duclos@stesio.jol";"0459000393"
5 "Enzo";"HAMEL";"HAMEL Enzo";"enzo.hamel@stesio.jol";"0459000403"
```

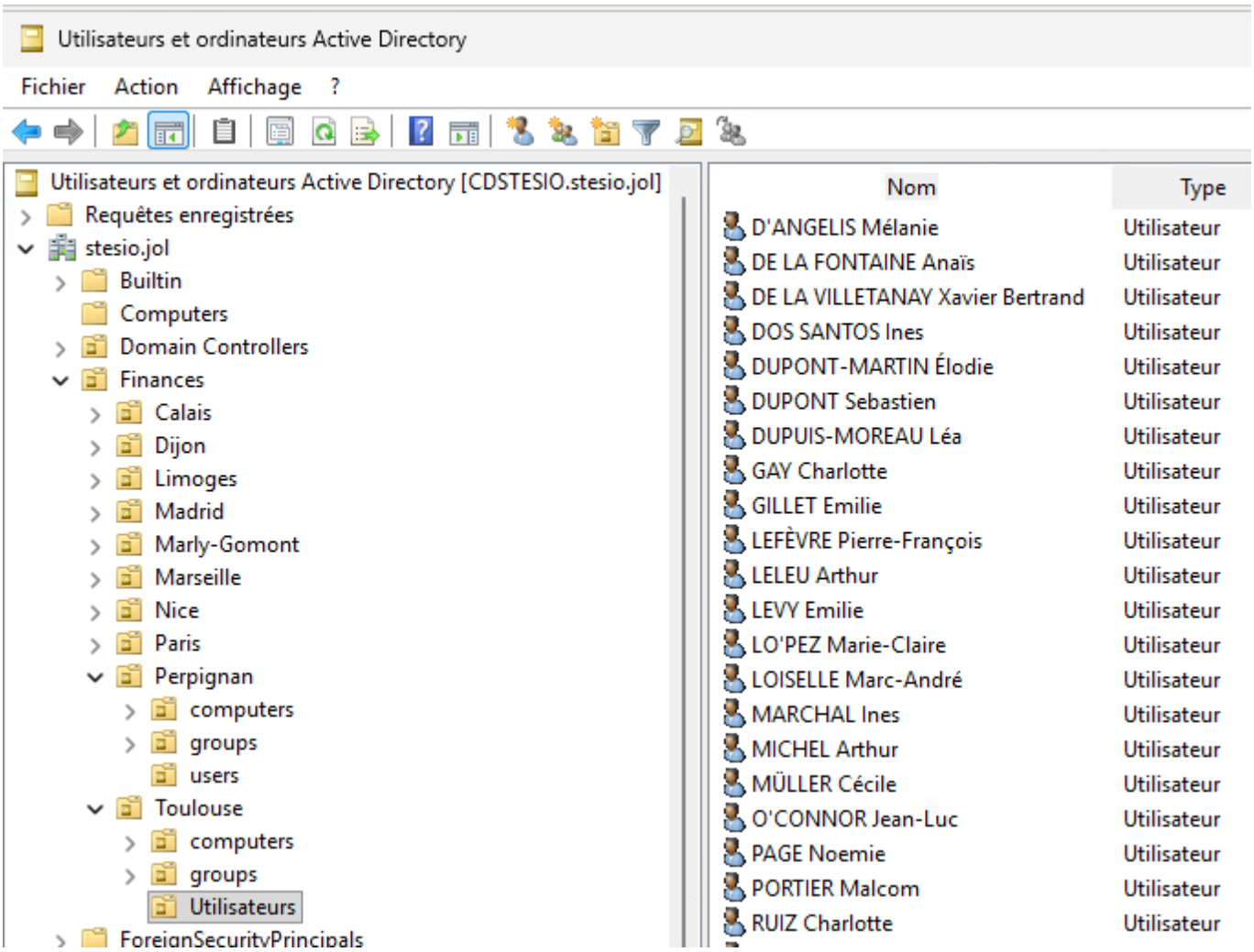
Figure 3: Extrait du contenu du fichier usersPerpignan.csv

Livrables

voir à la fin de cette étape

Phase 5.3 : Extraction des informations de l'OU Toulouse

On souhaite adapter le script utilisé pour l'OU **Perpignan** afin d'extraire les informations des utilisateurs de l'OU **Toulouse**.



À partir de la capture d'écran fournie et du script utilisé en Phase 5.2 (`extractUsersPerpignan.ps1`), adaptez ce script pour extraire les informations des utilisateurs de l'OU Toulouse.

- Nommez le nouveau script : `extractUsersToulouse.ps1`.
- Nommez le fichier CSV de sortie : `usersToulouse.csv`.
- Identifiez le chemin de l'OU qui contient les utilisateurs de Toulouse dans la structure Active Directory.
- Modifiez le script pour cibler cette OU et générer le fichier CSV correspondant.
- Testez et vérifiez que les données extraites correspondent bien aux utilisateurs de l'OU Toulouse.

Livrables pour l'étape 5

Phase 5.1 : Intégration d'une station Windows 10 au domaine

- Un compte-rendu incluant :
  - Un résumé des objectifs de la phase.
  - Une capture d'écran confirmant le renommage de la station (ex: `W10-NNNNN-P`).
  - Une capture d'écran de l'intégration réussie au domaine `stesio.jol`.
  - Une capture d'écran des résultats des commandes PowerShell :
    - `Test-NetConnection -ComputerName stesio.jol -Port 389` (résultat : `TcpTestSucceeded : True`).
    - `whoami /all` (affichant les informations du compte `binomeXX`).

Phase 5.2 : Extraction des informations de l'OU Perpignan

- Le script `extractUsersPerpignan.ps1` (commenté et documenté).
- Le fichier CSV généré : `usersPerpignan.csv`.
- Un compte-rendu incluant :
  - Un résumé des objectifs de la phase.
  - Une capture d'écran du contenu du fichier `usersPerpignan.csv`.
  - Une capture d'écran de l'exécution réussie du script.
  - Une explication des résultats obtenus (ex: nombre d'utilisateurs extraits, vérification des colonnes).

Phase 5.3 : Extraction des informations de l'OU Toulouse

- Le script `extractUsersToulouse.ps1` (commenté et documenté).
- Le fichier CSV généré : `usersToulouse.csv`.

Étape 06 - Renforcement de la politique de mots de passe

Phase 6.1 : Génération de mots de passe sécurisés avec un tableur

La société **STESIO** souhaite renforcer sa politique de mots de passe après plusieurs cyberattaques. Dans cette première phase, chaque mot de passe devra respecter les **critères de sécurité suivants** :

- **Longueur minimale** : 10 caractères.
- **Contraintes** :
  - Au moins **1 majuscule**, **1 minuscule** et **1 chiffre**.
  - **Exclusion des caractères** : espace, lettre **O majuscule** et chiffre **0** (pour éviter toute confusion entre "O" et "0").
- **Structure imposée** :

  Le mot de passe sera construit selon le modèle suivant : **C1 C2 C3 C4 C5 C6 C7 C8 C9 C10**
  - **C1** : lettre minuscule
  - **C2** : lettre majuscule
  - **C3** : chiffre
  - **C4** : lettre minuscule
  - **C5** : lettre minuscule
  - **C6** : chiffre
  - **C7** : chiffre
  - **C8** : lettre majuscule
  - **C9** : lettre minuscule
  - **C10** : lettre majuscule

Consignes :

1. Créez un fichier tableur nommé **`usersPassword`** (au format `.ods` ou `.xlsx`).
2. Utilisez des **formules** pour générer chaque caractère du mot de passe selon les contraintes définies.
3. Exportez le résultat dans un fichier **CSV** nommé **`usersPassword.csv`** avec une seule colonne intitulée **`password`**.

Livrables : *(voir à la fin de l'étape)*

Phase 6.2 : Génération de mots de passe robustes avec un script Python

Le service informatique décide de **durcir la politique de mots de passe** avec les contraintes suivantes :

- **Longueur minimale** : 12 caractères.
- **Contraintes** :
  - Au moins **2 majuscules**, **2 minuscules**, **2 chiffres** et **2 caractères spéciaux**.
  - **Exclusion des caractères** : espace, lettre **O majuscule** et chiffre **0**.
  - **Caractères spéciaux autorisés** : `! @ # $ % & * + - = . < > / ? :`
  - **Pas de mots issus d'un dictionnaire** (français, anglais, etc.).
  - **Pas de caractères spéciaux** en début ou fin de mot de passe.
  - **Pas de doublons** entre les mots de passe.

Consignes :

1. Créez un programme Python nommé **`createPassword.py`** qui génère un fichier CSV, **`usersPassword2.csv`**.
2. Le fichier CSV doit contenir une colonne intitulée **`password`** et autant de mots de passe que spécifié en argument.

Livrables : *(voir à la fin de l'étape)*

Phase 6.3 : Validation du respect des contraintes d'une liste de mots de passe

Le service informatique souhaite un programme capable de **vérifier la validité d'une liste de mots de passe** selon les contraintes de la **Phase 6.2** (hors vérification de dictionnaire).

Consignes :

1. Élaborez un programme Python nommé **`testPassword.py`**.
2. Le programme doit vérifier les mots de passe fournis dans un fichier CSV spécifié en argument.
3. Il doit indiquer si chaque mot de passe est valide ou signaler la nature du problème rencontré.
4. Vérifiez qu'il n'y a pas de doublons dans la liste.

Livrables pour l'étape 06

Phase 6.1 :

- Un compte-rendu incluant :
  - Un résumé des objectifs de la phase.
  - Une explication de la méthode utilisée pour générer les mots de passe (formules, logique).
  - Une capture d'écran du contenu du fichier `usersPassword.csv`.
- Le fichier tableur : `usersPassword.ods` (Calc) ou `usersPassword.xlsx` (Excel).
- Le fichier CSV généré : `usersPassword.csv`.

Phase 6.2 :

- Un compte-rendu incluant :
  - Un résumé des objectifs de la phase.
  - Une capture d'écran du contenu des fichiers `createPassword.py` et `usersPassword2.csv`.
- Le programme Python commenté : `createPassword.py`.
- Le fichier généré : `usersPassword2.csv`.

Phase 6.3 :

- Un compte-rendu incluant :
  - Un résumé des objectifs de la phase.
  - Une capture d'écran du test du programme Python commentée.
- Le programme Python commenté : `testPassword.py`.
- Le fichier d'entrée CSV utilisé pour les tests.

Étape 07 - Génération des logins utilisateurs pour MariaDB

Contexte et objectifs :

La société **STESIO** souhaite préparer les données nécessaires à la création des comptes utilisateurs et de leurs bases de données sur le serveur **MariaDB** (Rocky 9.5). Cette étape vise à **transformer les données des utilisateurs** (issues de `usersToulouse.csv` et `usersPassword2.csv`) en un fichier structuré contenant les logins, mots de passe, noms et prénoms normalisés.

Consignes :

1. **Utilisez les fichiers suivants** :
   - `usersToulouse.csv` (liste des utilisateurs extraits de l'Active Directory).
   - `usersPassword2.csv` (liste des mots de passe sécurisés générés précédemment).
2. **Développez le programme Python** `genererLoginToulouse.py` qui :
   - Lit les fichiers CSV et extrait les informations nécessaires.
   - Nettoie et normalise les noms d'utilisateurs en s'aidant de la logique du script `loginNormaliser.py`.
   - Génère un fichier `loginToulouse.csv` contenant les champs suivants :
     - **`login`** (format : `pnom`, première lettre du prénom suivie du nom en minuscules).
     - **`password`** (issu de `usersPassword2.csv`).
     - **`lastname`** (nom de famille).
     - **`firstname`** (prénom).

Exemple de contenu du fichier `loginToulouse.csv` :

```
login;password;lastname;firstname
alambert;MotDePasse123;LAMBERT;Alain
bmorane;AutreMotDePasse456;MORANE;Bob
```

Livrables :

- Un compte-rendu incluant :
  - Un résumé des objectifs de l'étape.
  - Une explication de la méthode utilisée pour nettoyer et normaliser les noms d'utilisateurs.
  - Des captures d'écran des fichiers `usersToulouse.csv`, `usersPassword2.csv`, et `loginToulouse.csv`.
- Le programme Python commenté : `genererLoginToulouse.py`.
- Le fichier généré : `loginToulouse.csv`.

Étape 08 - Création du script SQL pour utilisateurs et bases MariaDB

Contexte et objectifs :

La société **STESIO** souhaite automatiser la création des comptes utilisateurs et de leurs bases de données sur le serveur **MariaDB** (Rocky 9.5). Cette étape vise à **générer un script SQL** à partir des données transformées (`loginToulouse.csv`), puis à **créer les utilisateurs et leurs bases de données** avec les privilèges appropriés.

Consignes :

1. **Utilisez le fichier suivant** :
   - `loginToulouse.csv` (fichier contenant les logins, mots de passe, noms et prénoms normalisés).
2. **Développez le programme Python** `genererScriptSqlCreate.py` qui :
   - Lit le fichier `loginToulouse.csv` et extrait les informations nécessaires.
   - Génère un script SQL (`creerUsersBddAcces.sql`) contenant les commandes pour :
     - Créer les **utilisateurs MariaDB** (format : `pnom`).
     - Créer les **bases de données associées** (format : `st_pnom`). *(Le préfixe `st_` permet d'identifier clairement les bases associées à la société **STESIO** lors d'un déploiement sur un serveur mutualisé.)*

       Exemple : Pour **Bob MORANE**, la base de données sera nommée `st_bmorane`.
     - Attribuer les **privilèges nécessaires** sur la base de données de l'utilisateur :
       - Créer, modifier et supprimer des tables et des colonnes.
       - Insérer, modifier et supprimer des données.
       - Interroger sa base de données.
3. **Exécutez le script SQL généré** (`creerUsersBddAcces.sql`) sur votre serveur MariaDB (Rocky 9.5).
4. **Testez les fonctionnalités** pour **2 utilisateurs choisis aléatoirement** :
   - Se connecter à sa base de données avec le compte utilisateur et le mot de passe généré.
   - **Vérifier que l'utilisateur ne voit que sa propre base de données** (ex: `st_pnom`) et pas celles des autres utilisateurs.
   - Créer une table dans sa base de données.
   - Créer des colonnes dans cette table et ajouter 2 enregistrements.
   - Récupérer des informations de cette table avec une requête.
   - Modifier un enregistrement, puis le supprimer.
   - Ajouter une colonne dans la table, puis la supprimer.
   - Supprimer la table.

Livrables :

- Un compte-rendu incluant :
  - Un résumé des objectifs de l'étape.
  - Des captures d'écran des fichiers `loginToulouse.csv`, `genererScriptSqlCreate.py` et `creerUsersBddAcces.sql`.
  - Des captures d'écran commentées des tests réalisés sur le serveur MariaDB (connexion, création de tables, manipulations des données, etc.).
- Le programme Python commenté : `genererScriptSqlCreate.py`.
- Le script SQL généré : `creerUsersBddAcces.sql`.

Étape 09 - Création du script SQL de suppression utilisateurs et BDD

Contexte et objectifs :

La société **STESIO** souhaite automatiser la **suppression des comptes utilisateurs et de leurs bases de données** sur le serveur **MariaDB** (Rocky 9.5). Cette étape vise à **générer un script SQL** à partir d'un fichier dédié (`loginToulouseDelete.csv`), contenant uniquement les comptes à supprimer, puis à **supprimer les utilisateurs et leurs bases de données** de manière sécurisée.

Note : Pour cette étape, vous devez **copier le fichier `loginToulouse.csv` en `loginToulouseDelete.csv`** et **conserver uniquement les comptes que vous souhaitez supprimer** (par exemple, les 5 premiers comptes). Copier toute la ligne depuis le fichier source est plus rapide et évite les erreurs de saisie manuelle.

Consignes :

1. **Préparez le fichier `loginToulouseDelete.csv`** :
   - Copiez le fichier `loginToulouse.csv` en `loginToulouseDelete.csv`.
   - **Conservez uniquement les lignes des comptes à supprimer** (exemple : les 5 premiers comptes).
2. **Développez le programme Python** `genererScriptSqlDrop.py` qui :
   - Lit le fichier `loginToulouseDelete.csv` et extrait les informations nécessaires.
   - **Les champs inutiles (`password`, `firstname`, et `lastname`) seront ignorés par le script.**
   - Génère un script SQL (`supprimerUsersBddAcces.sql`) contenant les commandes pour :
     - Supprimer les **bases de données associées** (format : `st_pnom`).
     - Supprimer les **utilisateurs MariaDB** (format : `pnom`).
3. **Exécutez le script SQL généré** (`supprimerUsersBddAcces.sql`) sur votre serveur MariaDB (Rocky 9.5).
4. **Vérifiez que les utilisateurs et leurs objets ont disparu** :
   - Vérifiez que les bases de données et les utilisateurs n'apparaissent plus dans le SGBD.
   - Assurez-vous que les comptes supprimés ne peuvent plus se connecter.
   - **Testez spécifiquement la suppression des 5 premiers comptes** pour valider le processus.

Livrables :

- Un compte-rendu incluant :
  - Un résumé des objectifs de l'étape.
  - Une explication de la méthode utilisée pour sélectionner les comptes à supprimer.
  - Des captures d'écran des fichiers `loginToulouseDelete.csv`, `genererScriptSqlDrop.py` et `supprimerUsersBddAcces.sql`.
  - Des captures d'écran commentées des tests réalisés sur le serveur MariaDB (vérification de la suppression des bases de données et des utilisateurs).
- Le programme Python commenté : `genererScriptSqlDrop.py`.
- Le script SQL généré : `supprimerUsersBddAcces.sql`.
- Le fichier `loginToulouseDelete.csv` (contenant uniquement les comptes à supprimer).

Étape 10 - Rédaction d'un mode opératoire pour l'administrateur système

Contexte et objectifs :

La société **STESIO** souhaite documenter les procédures de **gestion automatisée des utilisateurs et de leurs bases de données MariaDB**. Cette étape vise à **rédiger un mode opératoire clair et précis**, destiné à l'administrateur système.

Consignes :

Rédigez un **mode opératoire structuré**, incluant :

- Les **étapes d'extraction** des utilisateurs depuis l'Active Directory.
- Les **instructions pour exécuter les programmes Python** (transformation des données, génération des scripts SQL).
- Les **procédures d'application des scripts SQL** sur le serveur MariaDB.
- Les **contrôles post-opération** pour valider le bon déroulement des actions.

Livrables :

- **Un compte-rendu** incluant :
  - Un **résumé des objectifs** de l'étape.
  - Le **mode opératoire** (format PDF ou document structuré).
  - Des **captures d'écran** illustrant les étapes clés (exécution des scripts, vérifications).

Étape 11 - Création de la base de données centralisée pour les RH

Contexte et Objectifs :

La société **STESIO** souhaite que les **Ressources Humaines (RH)** gèrent les utilisateurs, leurs bases de données, comptes et mots de passe sur le serveur **MariaDB**. Les informations doivent être stockées dans une base de données centralisée, **`db_toulouse`**.

Les utilisateurs doivent pouvoir accéder uniquement aux informations suivantes :

- **Nom**
- **Prénom**
- **Adresse email**

Les autres informations (comme les **logins**, **numéros de téléphone** et **mots de passe**) doivent rester **confidentielles**.

Consignes :

1. **Concevoir le schéma relationnel :**
   - Créez **deux tables** dans la base de données **`db_toulouse`** :
     - **`user_public`** : Contiendra les informations accessibles aux utilisateurs (nom, prénom, email).
     - **`user_private`** : Contiendra les informations confidentielles (login, numéro de téléphone, mot de passe haché).
   - Utilisez une relation **one-to-one** entre les deux tables.
2. **Importer les données depuis les fichiers CSV :**
   - **Phase 1 : Remplissage de la table `user_public`**
     - Utilisez le fichier **`usersToulouse.csv`** pour importer les informations publiques (nom, prénom, email).
     - Utilisez la commande **`LOAD DATA INFILE`** pour importer ces données.
     - **Remarque :**
       - Assurez-vous que le fichier CSV est **accessible** depuis le serveur MariaDB.
       - Si les colonnes du CSV ne correspondent pas exactement aux noms des champs dans la table, utilisez des **variables temporaires** (`@dummy`) pour ignorer les colonnes non utilisées.
       - Pour un exemple d'importation, reportez-vous à l'**Annexe 2**.

- **Phase 2 : Remplissage de la table `user_private`**
  - Utilisez les fichiers **`usersToulouse.csv et loginToulouse.csv`** pour importer les informations confidentielles (login, numéro de téléphone, mot de passe). Chaque utilisateur doit avoir un mot de passe différent.
  - Le **login** et le **numéro de téléphone** doivent être intégrés dans cette table.
  - Le **mot de passe** doit être **haché** avant d'être enregistré dans la base de données. Pour cette étape, utilisez la fonction **`PASSWORD()`** de MariaDB, bien qu'elle soit **obsolète et peu sécurisée** (double hachage SHA-1). Cette solution est **mieux que rien** et sera améliorée lors d'une prochaine étape.
  - **Remarque :**
    - Assurez-vous que les logins, numéros de téléphone et mots de passe sont correctement associés aux utilisateurs.
    - Pour un exemple d'importation, reportez-vous à l'**Annexe 2**.

3. **Créer l'utilisateur RH :**
   - Créez un utilisateur nommé **`rhtoulouse`** avec les droits nécessaires pour gérer la base de données **`db_toulouse`**.
   - Cet utilisateur doit pouvoir : **Consulter, Ajouter, Modifier, Supprimer** les données.
   - Il ne doit **pas** avoir le droit de modifier la structure de la base (comme créer ou supprimer des tables).
4. **Tests :**
   - Vérifiez que :
     - L'utilisateur **`rhtoulouse`** peut effectuer les opérations autorisées.
     - Les mots de passe sont bien **hachés** dans la table `user_private`.

Livrables :

- Un **compte-rendu** incluant :
  - Un **résumé des objectifs** de l’étape.
  - Le **schéma relationnel** de la base de données **`db_toulouse`**.
  - Le **script SQL** pour créer la base de données et ses tables.
  - Les **captures d’écran** des tests réalisés :
    - Création de la base et des tables.
    - Importation des données depuis les fichiers CSV.
    - Vérification des droits d’accès pour **`rhtoulouse`**.
    - Vérification que les mots de passe sont bien **hachés** dans la table `user_private`.

Étape 12 - Automatisation de l'attribution des droits aux utilisateurs Toulouse

Contexte et Objectifs :

Les utilisateurs **Toulouse** (créés précédemment lors de l'étape **"Création du script SQL pour utilisateurs et bases MariaDB"**) doivent pouvoir **consulter uniquement** la table contenant les informations suivantes : Nom, Prénom, Adresse email.

Consignes :

1. **Proposer un script Python :**
    - Développez un script nommé **`genererScriptSqlDroitsToulouse.py`** pour générer un script SQL permettant d'ajouter les droits de consultation à **tous les utilisateurs Toulouse** de manière automatisée.
    - Ce script doit :
        - Lire la liste des utilisateurs (pnom) depuis le fichier **`loginToulouse.csv`**.
        - Générer un script SQL nommé **`attribuerDroitsSelectToulouse.sql`**, qui accorde les droits de **consultation** sur la table **`user_public`** à chaque utilisateur.
2. **Tests :**
    - Vérifiez que :
        - Les utilisateurs **Toulouse** ne peuvent accéder qu'aux informations autorisées.

Livrables :

- Un **compte-rendu** incluant :
    - Un **résumé des objectifs** de l'étape
    - Le **script Python** : **`genererScriptSqlDroitsToulouse.py`**.
    - Le **script SQL généré** : **`attribuerDroitsSelectToulouse.sql`**.
    - Les **captures d'écran** des tests réalisés :
        - Vérification des droits d'accès pour les utilisateurs **Toulouse**.

Étape 13 - Renforcement de la sécurité des mots de passe dans la base de données

Contexte et Objectifs :

La sécurité des mots de passe est un enjeu majeur pour protéger les données sensibles. Les algorithmes de hachage obsolètes, comme **SHA-1** ou la fonction `PASSWORD()` de MariaDB, ne sont plus considérés comme sûrs face aux attaques par force brute ou par dictionnaire. Il est donc essentiel d'utiliser des **algorithmes de hachage modernes et robustes**, comme **bcrypt**, qui intègrent un **salt** unique et un **coût de calcul élevé** pour résister à ces attaques.

Pourquoi renforcer la sécurité des mots de passe ?

- Les algorithmes modernes comme **bcrypt** sont conçus pour être résistants aux attaques par force brute.
- Ils intègrent un **salt** automatique pour chaque mot de passe, ce qui empêche les attaques par table arc-en-ciel.
- Ils sont largement recommandés par les experts en sécurité pour stocker les mots de passe de manière sécurisée.

Dans cette étape, vous allez :

- **Remplacer les anciens hachages** des mots de passe par des hachages **bcrypt**.
- **Mettre à jour la base de données** pour stocker ces nouveaux hachages.

Phase 13.1 : Création d'un script Python pour hacher les mots de passe avec bcrypt

1. **Présentation de la solution :**
   - Utilisez la bibliothèque **`bcrypt`** en Python, qui est l'une des méthodes les plus sûres pour hacher les mots de passe.
2. **Écrire un script Python :**
   - Développez un script nommé **`generateSecurePasswords.py`**.
   - Ce script doit :
     - Lire les logins, mots de passe en clair, noms et prénoms depuis le fichier **`loginToulouse.csv`**.
     - Hacher chaque mot de passe avec **bcrypt** (en utilisant un coût adapté, par exemple 12).
     - Stocker les nouveaux hachages dans un fichier nommé **`secureUsersPasswords.csv`**, avec le format suivant :

       ```
       login;password_bcrypt;password;lastname;firstname
       ```

Phase 13.2 : Mise à jour de la base de données

1. **Création du script SQL `updatePasswordsBcrypt.sql` :**
   - **Votre tâche** : Écrivez un script SQL nommé **`updatePasswordsBcrypt.sql`** qui :
     - Ajoute une nouvelle colonne **`password_bcrypt`** à la table **`user_private`** (si elle n'existe pas déjà).
     - Met à jour la colonne **`password_bcrypt`** avec les nouveaux hachages bcrypt, en utilisant les données du fichier **`secureUsersPasswords.csv`**.
     - Supprime l'ancienne colonne **`password`** (contenant les hachages obsolètes) une fois la mise à jour terminée.
2. **Exécution du script :**
   - Exécutez le script **`updatePasswordsBcrypt.sql`** sur votre serveur MariaDB pour appliquer les modifications.

Phase 13.3 : Vérifications

- **Vérification visuelle des hachages bcrypt :**
  - **Votre tâche** : Trouvez et exécutez une requête SQL qui permet d'afficher les 3 premiers enregistrements de la table **`user_private`**, en incluant les colonnes **`login`** et **`password_bcrypt`**.
  - **Indice** : Utilisez une requête de type `SELECT` avec une clause `LIMIT`.
  - **À vérifier** : Les hachages bcrypt commencent toujours par **`$2b$`** (ou parfois `$2a$` ou `$2y$`), suivi du coût (par exemple, `12`), puis d'une chaîne de caractères aléatoires.

Livrables :

- Un **compte-rendu** incluant :
  - **Un résumé des objectifs de l'étape**.
  - Une **explication de l'intérêt de bcrypt** et des risques liés aux anciens algorithmes.
  - Le **script Python** : **`generateSecurePasswords.py`**.
  - Le **fichier généré** : **`secureUsersPasswords.csv`**.
  - Le **script SQL** : **`updatePasswordsBcrypt.sql`**.
  - Les **captures d'écran** des tests réalisés :
    - Résultat de la requête SQL affichant les 3 premiers enregistrements de la table **`user_private`** (colonnes **`login`** et **`password_bcrypt`**).
    - Vérification visuelle que les hachages bcrypt commencent bien par **`$2b$`**.

Étape 14 - Validation de l'authentification des utilisateurs Toulouse

Contexte et Objectifs :

Après avoir renforcé la sécurité des mots de passe en utilisant **bcrypt**, il est essentiel de **valider que les utilisateurs peuvent toujours s'authentifier correctement** avec leurs nouveaux mots de passe hachés. Cette étape permet de s'assurer que le processus de hachage et de mise à jour des mots de passe n'a pas introduit d'erreurs, et que les utilisateurs peuvent toujours accéder à leurs comptes de manière sécurisée.

Pourquoi valider l'authentification ?

- Vérifier que les hachages **bcrypt** stockés dans la base de données (ou dans le fichier CSV) correspondent bien aux mots de passe saisis par les utilisateurs.
- S'assurer que le processus de hachage et de comparaison fonctionne correctement avant de déployer la solution en production.
- Permettre de comprendre comment fonctionne l'authentification sécurisée avec **bcrypt**.

Consignes :

1. **Présentation de la solution :**
    - Vous allez créer un script Python qui simule le processus d'authentification d'un utilisateur.
    - Le script va lire les hachages **bcrypt** stockés dans le fichier **`secureUsersPasswords.csv`**, puis comparer le hachage du mot de passe saisi par l'utilisateur avec celui stocké dans le fichier.
2. **Écrire un script Python :**
    - Développez un script nommé **`checkUserAuthentication.py`**.
    - Ce script doit :
        - Lire le fichier **`secureUsersPasswords.csv`** pour récupérer les logins et les hachages **bcrypt**.
        - Demander à l'utilisateur de saisir un **login** (parmi les utilisateurs Toulouse).
        - Demander à l'utilisateur de saisir son **mot de passe en clair**.
        - Hacher le mot de passe saisi avec **bcrypt** (en utilisant le même coût que celui utilisé pour générer les hachages).
        - Comparer le hachage généré avec celui stocké dans le fichier **`secureUsersPasswords.csv`**.
        - Afficher un message indiquant si l'authentification a **réussi** ou **échoué**.

Livrables :

- Un **compte-rendu** incluant :
    - **Un résumé des objectifs de l'étape**.
    - Le **script Python** : **`checkUserAuthentication.py`** (commenté).
    - Les **captures d'écran** des tests réalisés :
        - Exemple d'exécution réussie du script (authentification réussie).
        - Exemple d'exécution échouée du script (authentification échouée).

Étape 15 - Évolution 1 – Paramètres en ligne de commande

Contexte et Objectifs :

La société STESIO souhaite **automatiser et flexibiliser** la génération des scripts SQL pour la création des utilisateurs et de leurs bases de données. Actuellement, les noms des fichiers d'entrée et de sortie sont codés en dur dans le script Python. Pour rendre le programme plus polyvalent, il est nécessaire de permettre à l'administrateur de **spécifier les noms des fichiers en ligne de commande**.

Pourquoi cette évolution ?

- Permettre une **utilisation plus flexible** du script pour différents jeux de données.
- Faciliter l'intégration du script dans des **processus automatisés** ou des scripts shell.
- Préparer le terrain pour des évolutions futures, comme la gestion de plusieurs environnements (test, production).

Consignes :

1. Reprendre les fonctionnalités du programme `genererScriptSqlCreate.py`.
2. Créer une nouvelle version du programme Python, nommée **genererScriptSqlCreate_etape15.py**, qui accepte les paramètres suivants en ligne de commande :
    - Le nom du fichier des utilisateurs (ex: `usersToulouse.csv`).
    - Le nom du fichier résultat (ex: `creerUsersBddAcces.sql`).
3. **Exemple d'utilisation :**

```
python3 genererScriptSqlCreate_etape15.py usersToulouse.csv creerUsersBddAcces.sql
```

Livrables :

- Un compte-rendu incluant :
    - Un résumé des objectifs de l’étape.
    - Une nouvelle version du mode opératoire avec captures d’écran.
    - Une capture d’écran du contenu du fichier `genererScriptSqlCreate_etape15.py`.
- Le programme Python commenté : `genererScriptSqlCreate_etape15.py`.

Étape 16 - Évolution 2 – Mise en œuvre de "patterns" pour créer ou supprimer les utilisateurs

Contexte et Objectifs :

L'administrateur système souhaite **simplifier la gestion** des scripts SQL pour la création et la suppression des utilisateurs et de leurs bases de données. Actuellement, il existe deux programmes distincts pour ces tâches. L'objectif est de **fusionner ces fonctionnalités** dans un seul programme, en utilisant des **fichiers de patterns** (modèles) pour générer les scripts SQL.

Pourquoi utiliser des patterns ?

- **Centraliser la logique** de génération des scripts SQL dans un seul programme.
- **Faciliter la maintenance** : Les fichiers de patterns peuvent être modifiés sans toucher au code Python.
- **Réduire les risques d'erreurs** en utilisant des modèles standardisés.

Consignes :

1. **Reprendre les fonctionnalités du programme `genererScriptSqlCreate_etape15.py`.**
2. **Créer une nouvelle version du programme Python**, nommée **genererScriptSql_etape16.py**, qui :
   - Génère les ordres SQL pour chaque utilisateur à partir d'un **fichier de pattern** (modèle) en format CSV.
   - Accepte les paramètres suivants en ligne de commande :
     - Le nom du fichier de pattern (ex: `patternSqlCreate.csv` ou `patternSqlDrop.csv`).
     - Le nom du fichier des utilisateurs (ex: `usersToulouse.csv`).
     - Le nom du fichier de script SQL résultant (ex: `creerUsersBddAcces.sql` ou `supprimerUsersBddAcces.sql`).

Explication des fichiers de pattern :

1. Fichier de pattern pour la création (`patternSqlCreate.csv`)

- Ce fichier contient des **commandes SQL génériques** pour créer un utilisateur, sa base de données et ses privilèges.
- **Exemple de contenu du fichier `patternSqlCreate.csv` :**

```
CREATE USER 'XXXXX'@'localhost' IDENTIFIED BY 'PPPPP';
CREATE DATABASE db_XXXXX;
GRANT ALL PRIVILEGES ON db_XXXXX.* TO 'XXXXX'@'localhost';
```

  - **XXXXX** est un **motif** qui sera remplacé par le **login de l'utilisateur** (ex: `alambert`).
  - **PPPPP** est un **motif** qui sera remplacé par le **mot de passe sécurisé** de l'utilisateur.

  Exemple de script SQL généré :

```
CREATE USER 'alambert'@'localhost' IDENTIFIED BY
'MotDePasseSecurise123';
CREATE DATABASE db_alambert;
GRANT ALL PRIVILEGES ON db_alambert.* TO 'alambert'@'localhost';
```

```sql
CREATE USER 'bmorane'@'localhost' IDENTIFIED BY
'AutreMotDePasseSecurise456';
CREATE DATABASE db_bmorane;
GRANT ALL PRIVILEGES ON db_bmorane.* TO 'bmorane'@'localhost';
```

2. Fichier de pattern pour la suppression (`patternSqlDrop.csv`)

- Ce fichier contient des **commandes SQL génériques** pour supprimer un utilisateur et sa base de données.

Livrables :

- Un compte-rendu incluant :
  - Un résumé des objectifs de l'étape.
  - Une nouvelle version du mode opératoire avec captures d'écran.
  - Des captures d'écran du contenu des fichiers `patternSqlCreate.csv`, `patternSqlDrop.csv`, `genererScriptSql_etape16.py`, et des scripts SQL générés.
- Le programme Python commenté : `genererScriptSql_etape16.py`.
- Les fichiers de pattern : `patternSqlCreate.csv` et `patternSqlDrop.csv`.

Étape 17 - Évolution 3 – Exécution automatique du script SQL

Contexte et Objectifs :

Pour **simplifier le processus** de gestion des utilisateurs, il est souhaitable d'éviter les manipulations manuelles nécessaires pour exécuter les scripts SQL (comme le démarrage d'un client MariaDB). L'objectif est d'**automatiser l'exécution** des scripts SQL directement depuis le programme Python.

Pourquoi cette évolution ?

- **Gagner du temps** en évitant les étapes manuelles.
- **Réduire les risques d'erreurs** liées à l'exécution manuelle des scripts.
- **Faciliter l'intégration** dans des processus automatisés ou des pipelines CI/CD.

Consignes :

1. **Reprendre les fonctionnalités du programme genererScriptSql_etape16.py.**
2. **Créer une nouvelle version du programme Python, nommée genererScriptSql_etape17.py,** qui :
    - Génère le script SQL comme précédemment.
    - **Exécute automatiquement** le script SQL généré sur le serveur MariaDB.
3. **Exemple d'utilisation :**

```
python3 genererScriptSql_etape17.py patternSqlCreate.csv usersToulouse.csv
```

4. **Recherches nécessaires :**
    - Comment exécuter un script SQL depuis une ligne de commande Linux ?
    - Comment exécuter une commande Linux depuis un script Python ?

Livrables :

- Un **compte-rendu** incluant :
    - Un **résumé des objectifs de l'étape**.
    - Une **nouvelle version du mode opératoire** avec captures d'écran.
    - Des **captures d'écran du contenu des fichiers CSV, Python et SQL**.
- Le **programme Python commenté** : **`genererScriptSql_etape17.py`**.

Étape 18 - Évolution 4 – Exécution directe des requêtes SQL depuis Python

Contexte et Objectifs :

Pour **aller plus loin dans l'automatisation**, il est souhaitable de permettre au programme Python d'exécuter **directement les requêtes SQL** sur le SGBD MariaDB, sans passer par un fichier script intermédiaire. Cela permet de **simplifier le processus** et de **mémoriser les requêtes exécutées** pour un suivi ou un audit.

Pourquoi cette évolution ?

- **Supprimer l'étape intermédiaire** de génération et d'exécution d'un fichier script SQL.
- **Centraliser les logs** des requêtes exécutées pour un suivi ou un audit.
- **Améliorer la robustesse** en réduisant le nombre d'étapes et de fichiers intermédiaires.

Consignes :

1. **Recherches nécessaires :**
   - Quels modules Python permettent de se connecter et d'exécuter des requêtes SQL sur un SGBD MariaDB ?
   - Comment mémoriser les requêtes exécutées dans un fichier texte ?
2. **Reprendre les fonctionnalités du programme `genererScriptSql_etape17.py`.**
3. **Créer une nouvelle version du programme Python**, nommée **genererScriptSql_etape18.py**, qui :
   - Exécute **directement les requêtes SQL** dans le SGBD MariaDB.
   - Mémorise les requêtes exécutées dans un fichier texte nommé automatiquement avec la date et l'heure courante (ex: `ordres_sql_2025_12_19__14_30.txt`).
4. **Exemple d'utilisation :**

```
python3 genererScriptSql_etape18.py patternSqlCreate.csv usersToulouse.csv
```

Livrables :

- Un compte-rendu incluant :
  - Un résumé des objectifs de l’étape.
  - Une nouvelle version du mode opératoire avec captures d’écran.
  - Des captures d’écran du contenu des fichiers CSV, Python et texte.
- Le programme Python commenté : `genererScriptSql_etape18.py`.
- Les fichiers texte générés par les jeux d'essai (création, suppression).

Étape 19 - Évolution 5 – Mise en œuvre d'une interface graphique

Contexte et Objectifs :

Pour **faciliter l'utilisation** du programme par les administrateurs système, il est souhaitable de mettre en place une **interface graphique**. Cela permettra de saisir ou de choisir les fichiers nécessaires et de lancer le traitement en un clic, sans avoir à se souvenir des paramètres en ligne de commande.

Pourquoi cette évolution ?

- **Rendre le programme plus accessible** aux utilisateurs non techniques.
- **Réduire les risques d'erreurs** liées à la saisie manuelle des paramètres.
- **Améliorer l'expérience utilisateur** avec une interface intuitive.

Consignes :

1. **Recherches nécessaires :**
   - Comment utiliser la bibliothèque **`tkinter`** pour créer une interface graphique en Python ?
   - Comment intégrer les fonctionnalités existantes dans une interface graphique ?
2. **Reprendre les fonctionnalités du programme genererScriptSql_etape18.py.**
3. **Créer une nouvelle version du programme Python**, nommée **genererScriptSql_etape19.py**, qui :
   - Propose une **interface graphique** pour saisir ou choisir :
     - Le nom du fichier de pattern.
     - Le nom du fichier des utilisateurs.
     - Le nom du fichier de script SQL généré.
   - Inclut un **bouton pour lancer le traitement**.
   - Inclut un **bouton pour quitter l'interface**.
4. **Exemple d'utilisation :**
   `genererScriptSql_etape19.py`

Livrables :

- Un compte-rendu incluant :
  - Un résumé des objectifs de l’étape.
  - Une nouvelle version du mode opératoire avec captures d’écran.
  - Des captures d’écran du contenu des fichiers CSV, Python et de l'interface graphique.
- Le programme Python commenté : `genererScriptSql_etape19.py`.

ANNEXE 1 - Notions de base sur Active Directory (AD) et les Unités d'Organisation (OU)

Pour bien comprendre ce que vous allez faire, voici quelques notions essentielles sur **Active Directory** et les **Unités d'Organisation (OU).**

Qu'est-ce qu'un annuaire Active Directory (AD) ?

- **Active Directory (AD)** est un service d'annuaire développé par Microsoft pour les environnements Windows.
- Il permet de **stocker et gérer des informations** sur les utilisateurs, les ordinateurs, les groupes, et d'autres objets dans un réseau.
- **Exemple** : Imaginez un annuaire téléphonique, mais pour un réseau informatique. Il permet de retrouver facilement des utilisateurs, des ordinateurs, et de gérer leurs droits d'accès.

À quoi sert Active Directory ?

- **Centraliser la gestion des utilisateurs et des ressources** : Ajouter, modifier ou supprimer des comptes utilisateurs, des groupes, et des ordinateurs.
- **Authentification et autorisation** : Permettre aux utilisateurs de se connecter avec un seul identifiant et mot de passe, et de contrôler leurs accès aux ressources.
- **Appliquer des politiques de sécurité** : Définir des règles pour les mots de passe, les droits d'accès, etc.

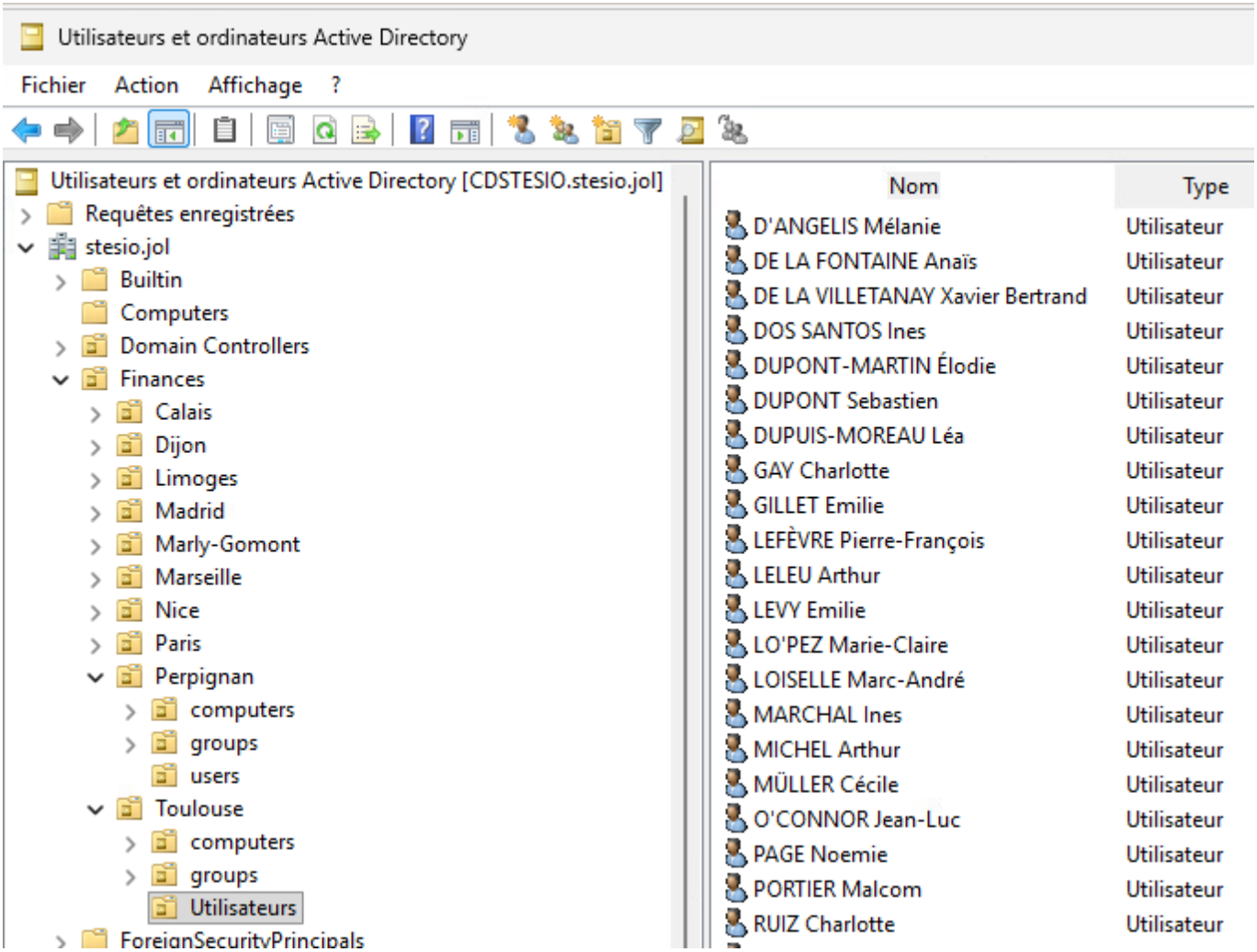
Qu'est-ce qu'une Unité d'Organisation (OU) ?

- Une **Unité d'Organisation (OU)** est un conteneur dans Active Directory qui permet de **regrouper des objets** (utilisateurs, ordinateurs, groupes) de manière logique.
- **Exemple** : Dans une entreprise, on peut avoir une OU pour chaque service (Comptabilité, Ressources Humaines, Informatique, etc.).
- **Pourquoi utiliser des OU ?**
  - **Organiser les objets** de manière hiérarchique.
  - **Appliquer des politiques de groupe** spécifiques à certains groupes d'utilisateurs ou d'ordinateurs.
  - **Déléguer des droits d'administration** à des niveaux spécifiques.

Structure d'Active Directory

- **Domaine** : Une unité logique qui regroupe des objets (ex: `stesio.jol`).
- **Arbre** : Un ensemble de domaines qui partagent une même structure hiérarchique.
- **Forêt** : Un ensemble d'arbres qui partagent un même schéma et une même configuration.
- **Contrôleur de domaine** : Un serveur qui gère les requêtes d'authentification et les modifications de l'annuaire.

Exemple concret



- **Domaine** : `stesio.jol`
- **Contrôleur de domaine** : CDSTESIO
- **OU** : `Finances` (pour le service comptabilité)
  - Sous-OU : `Toulouse` (pour les utilisateurs de la région Toulouse)
  - Sous-OU : `Perpignan` (pour les utilisateurs de la région Perpignan)
- **Utilisateurs** : Chaque utilisateur est stocké dans une OU en fonction de son service et de sa localisation.

Pourquoi est-ce important pour ce projet ?

- **Extraire des informations** : Vous allez utiliser des scripts PowerShell pour récupérer des informations sur les utilisateurs stockés dans des OU spécifiques.
- **Comprendre la structure** : Savoir où sont stockés les utilisateurs et comment y accéder est essentiel pour administrer un réseau.
- **Automatiser des tâches** : Les scripts PowerShell permettent d'automatiser des tâches répétitives, comme l'extraction d'informations sur les utilisateurs.

ANNEXE 2 - Exemple d'importation de données avec LOAD DATA INFILE

Contexte :

Vous disposez d'un fichier CSV nommé **`produits.csv`** contenant les informations suivantes sur des produits :

```
id_produit;nom_produit;prix;quantite;categorie;fournisseur
1;Stylo;1.50;100;Papeterie;FournisseurA
2;Cahier;3.20;50;Papeterie;FournisseurB
3;Souris;15.99;30;Informatique;FournisseurC
```

Vous souhaitez importer ces données dans une table SQL nommée **`produits`** avec la structure suivante :

```sql
CREATE TABLE produits (
    id_produit INT PRIMARY KEY,
    nom_produit VARCHAR(100) NOT NULL,
    prix DECIMAL(10, 2) NOT NULL,
    quantite INT NOT NULL,
    categorie VARCHAR(50),
    fournisseur VARCHAR(50)
);
```

Cas 1 : Noms de champs identiques entre le CSV et la table SQL

Si les noms des colonnes du CSV correspondent exactement aux noms des champs dans la table SQL, utilisez la commande suivante :

```sql
LOAD DATA INFILE '/chemin/vers/produits.csv'
INTO TABLE produits
FIELDS TERMINATED BY ';'  -- Séparateur de champs dans le CSV
LINES TERMINATED BY '\n'  -- Séparateur de lignes dans le CSV
IGNORE 1 ROWS             -- Ignorer la première ligne (en-tête)
(id_produit, nom_produit, prix, quantite, categorie, fournisseur);
```

Cas 2 : Noms de champs différents entre le CSV et la table SQL

Si les noms des colonnes du CSV **ne correspondent pas** aux noms des champs dans la table SQL, vous pouvez spécifier explicitement l'ordre des colonnes dans la commande `LOAD DATA INFILE`. Par exemple, si votre fichier CSV contient les colonnes suivantes :

```
ref;designation;tarif;stock;rayon;marque
1;Stylo;1.50;100;Papeterie;FournisseurA
2;Cahier;3.20;50;Papeterie;FournisseurB
3;Souris;15.99;30;Informatique;FournisseurC
```

Vous pouvez importer ces données dans la table **`produits`** en utilisant :

```
LOAD DATA INFILE '/chemin/vers/produits.csv'
INTO TABLE produits
FIELDS TERMINATED BY ';'  -- Séparateur de champs dans le CSV
LINES TERMINATED BY '\n'  -- Séparateur de lignes dans le CSV
IGNORE 1 ROWS             -- Ignorer la première ligne (en-tête)
(ref, designation, tarif, stock, rayon, marque);
```

Remarque :

- Les noms entre parenthèses dans la commande `LOAD DATA INFILE` doivent **correspondre aux noms des colonnes dans le fichier CSV**, pas aux noms des champs dans la table SQL.
- Assurez-vous que l'ordre des colonnes dans la commande correspond à l'ordre des colonnes dans le fichier CSV.

Cas 3 : Colonnes supplémentaires dans le CSV

Si votre fichier CSV contient des colonnes supplémentaires non présentes dans la table (par exemple, une colonne `date_ajout`), vous pouvez les ignorer en utilisant des **variables temporaires** (`@dummy`) :

```
LOAD DATA INFILE '/chemin/vers/produits.csv'
INTO TABLE produits
FIELDS TERMINATED BY ';'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(ref, designation, tarif, stock, rayon, marque, @dummy);
```

ANNEXE 3 - Extraits de contenu des différents fichiers CSV

Étape 02 - Lecture et écriture dans un fichier CSV en Python

Fichier : testEntree.csv

```
Prenom;NOM
Emile;ZOLA
Victor;HUGO
```

Fichier : testSortie.csv

```
login
ezola
vhugo
```

Étape 03 - Normalisation avancée des logins

Fichier : testEntreeSpeciaux.csv

```
Prenom;NOM
Alain;Lambert
Jean-Baptiste;Le Floc'h
```

Fichier : testSortieNormaliser.csv

```
login
alambert
jblefloch
```

Étape 05 - Intégration d'une station Windows 10 et extraction d'informations Active Directory

Fichier : usersPerpignan.csv

```
givenName;sn;displayName;mail;telephoneNumber
Julie;PRIGENT;PRIGENT Julie;julie.prigent@stesio.jol;0159000373
Caroline;PARMENTIER;PARMENTIER Caroline;caroline.parmentier@stesio.jol;0159000383
```

Fichier : usersToulouse.csv

```
givenName;sn;displayName;mail;telephoneNumber
Mélanie;D'ANGELIS;D'ANGELIS Mélanie;melanie.dangelis@stesio.jol;0159000403
Anaïs;DE LA FONTAINE;DE LA FONTAINE Anaïs;anais.delafontaine@stesio.jol;0159000413
```

Étape 06 - Renforcement de la politique de mots de passe

Fichier : usersPassword.csv

```
password
aB3dE7fG9h
kL5mN8pQ2r
```

Fichier : usersPassword2.csv

```
password
kL5@mN8#pQ2!
tU4$vW7%xY1*
```

Étape 07 - Génération des logins utilisateurs pour MariaDB

Fichier : loginToulouse.csv

```
login;password;lastname;firstname
alambert;kL5@mN8#pQ2!;LAMBERT;Alain
jblefloch;tU4$vW7%xY1*;LE FLOC'H;Jean-Baptiste
```